

Wallabot Documentation

Wallabot is an AI-powered C2C second-hand marketplace built with a FastAPI microservices backend, a Vue 3 frontend, and an LLM-driven agentic subsystem for intelligent product classification and price recommendations.

 [Download PDF](#)

Project Overview

- [Project Overview](#)
 - [What is Wallabot?](#)
 - [Sprint Deliverables](#)
 - [Key Features](#)
 - [Technology Stack](#)
 - [Repository Layout](#)
 - [Local Quick-Start](#)
 - [Design Principles](#)
- [System Architecture](#)
 - [Design Philosophy](#)
 - [High-Level Diagram](#)
 - [Service Responsibilities](#)
 - [Authentication Flow](#)
 - [Product State Machine](#)
 - [Wallet Ledger Architecture](#)
 - [Atomic Purchase Flow](#)
 - [Database Strategy](#)
 - [Frontend Architecture](#)
 - [Deployment Architecture \(Production\)](#)
 - [CI/CD Pipeline](#)

REST API Reference

- [REST API Reference](#)
 - [Auth Service API](#)
 - [Authentication](#)
 - [Health Check](#)

Project Overview

What is Wallabot?

Wallabot is a full-stack **C2C (consumer-to-consumer) second-hand marketplace** inspired by Wallapop. It allows registered users to list products, browse the catalogue, reserve items, and complete peer-to-peer purchases through an integrated digital wallet. The platform is augmented by an AI subsystem — the *Wallabot Agentic Service* — which provides intelligent product category suggestion and live market-based price estimation powered by LangChain LCEL, OpenAI GPT-4o-mini, and Tavily web search.

The project was developed across three sprints by team **BSPQ26-E9** as part of the Software Engineering course at the University of Deusto.

Sprint Deliverables

Sprint	Main deliverables
Sprint 1	User registration & login, product CRUD, basic catalogue browsing
Sprint 2	Digital wallet, product reservation with configurable timeout, atomic purchase flow, transaction history
Sprint 3	AI category suggestion agent, AI price recommendation agent, LangSmith observability tracing, user ratings system, public user profiles, full-text search on catalogue

Key Features

User Management & Authentication

- Email and password registration with **bcrypt** password hashing
- JWT-based stateless authentication (HS256) shared across all microservices via a single `SECRET_KEY` environment variable — no inter-service token validation call per request
- Social login via **Google OAuth 2.0** and **Facebook OAuth** using the `authlib` library
- Account linking: a social account is bound to one provider and rejects conflicting link attempts (you cannot link the same email to two different providers)
- Token payload: `{"sub": "<user_email>", "exp": <unix_timestamp>}`

System Architecture

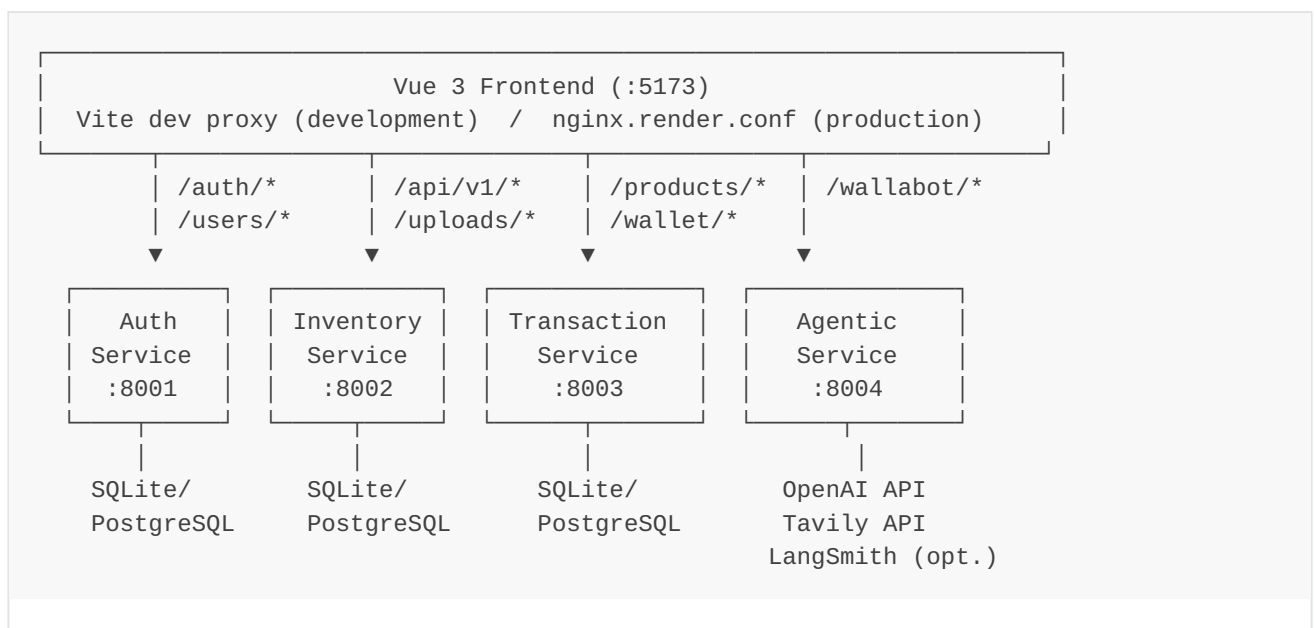
Design Philosophy

Wallabot follows a **microservices architecture** where each business domain is encapsulated in an independently deployable FastAPI service. Services share no databases and communicate only via HTTP REST. The Vue 3 frontend is the sole public entry point and routes every API call through a Vite proxy in development (or Nginx in production) to the appropriate service.

Key design decisions:

- **No shared database** — each service owns its schema; cross-service data access goes over HTTP with service-scoped JWTs
- **Stateless JWT validation** — every service validates tokens locally, eliminating a central auth round-trip per request
- **Append-only ledger** — wallet balance and product state history are immutable, audit-safe records
- **Graceful AI degradation** — agents always return a usable response even when the LLM provider is unreachable

High-Level Diagram



The Agentic Service has **no database** — every request is stateless and fully self-contained.

REST API Reference

Wallabot exposes its functionality through four independent HTTP REST services. This section documents every public endpoint, request/response schema, and error contract — the equivalent of an RMI interface specification.

All services accept and return **JSON** (`Content-Type: application/json`). Endpoints that require authentication expect a `Bearer` token in the `Authorization` header issued by the Auth Service.

- [Auth Service API](#)
 - [Authentication](#)
 - [Health Check](#)
 - [Registration & Login](#)
 - [Social Authentication \(OAuth\)](#)
 - [User Profiles](#)
 - [Ratings](#)
- [Inventory Service API](#)
 - [Health Check](#)
 - [Products](#)
 - [Image Uploads](#)
 - [ProductOut Schema](#)
- [Transaction Service API](#)
 - [Health Check](#)
 - [Wallet](#)
 - [Products \(Transaction View\)](#)
 - [Transactions](#)
 - [Error Reference](#)
 - [Reservation Cleanup Daemon](#)
- [Agentic Service API \(Wallabot\)](#)
 - [Health Check](#)
 - [Category Suggestion](#)
 - [Price Recommendation](#)
 - [LangSmith Tracing](#)

Auth Service API

Base URL (local): `http://localhost:8001` Base URL (production): configured via `AUTH_SERVICE_URL` environment variable

The Auth Service owns user identity: registration, login, social OAuth, JWT issuance, public user profiles, and the ratings system.

Authentication

Most endpoints in other services require a JWT access token issued by this service. Pass it as a Bearer token in every request:

```
Authorization: Bearer <access_token>
```

Tokens are signed with `HS256` using the shared `SECRET_KEY` environment variable and contain `{"sub": "<user_email>"}` as the payload subject.

Health Check

GET /health

Returns the service liveness status. Used by Docker healthcheck probes.

Response 200

```
{ "status": "ok" }
```

Inventory Service API

Base URL (local): `http://localhost:8002`

The Inventory Service owns the product catalogue: creating, reading, updating, and deleting listings, plus image uploads. Products stored here represent the *listing* view; the Transaction Service maintains a separate copy for commerce state.

Health Check

GET `/health`

```
{ "status": "ok" }
```

Products

GET `/api/v1/products`

Returns a list of all products, optionally filtered. No authentication required.

Query parameters

Parameter	Type	Description
<code>state</code>	<code>Available</code> <code>Reserved</code> <code>Sold</code>	Filter by availability state
<code>category</code>	string (min 1 char)	Filter by category name (case-insensitive)
<code>min_price</code>	float (≥ 0)	Minimum product price in EUR
<code>max_price</code>	float (≥ 0)	Maximum product price in EUR
<code>condition</code>	<code>New</code> <code>Like New</code> <code>Good</code> <code>Fair</code> <code>Poor</code>	Filter by item condition
<code>q</code>	string (min 1 char)	Full-text keyword search across title and description

Response 200 — array of `ProductOut`

Transaction Service API

Base URL (local): `http://localhost:8003`

The Transaction Service owns the commerce domain. It manages a digital wallet per user, coordinates product reservations (with automatic timeout expiry), and processes purchases via atomic wallet debit operations. It also exposes transaction history.

All endpoints require authentication (`Authorization: Bearer <token>`).

Health Check

GET `/health`

```
{ "status": "ok", "service": "transaction-service" }
```

Wallet

GET `/wallet/balance`

Returns the authenticated user's current wallet balance in EUR.

Response 200

```
{ "balance": 250.00 }
```

POST `/wallet/deposit`

Adds funds to the authenticated user's wallet.

Request body

Agentic Service API (Wallabot)

Base URL (local): `http://localhost:8004`

The Agentic Service is the AI subsystem of Wallabot. It provides two intelligent endpoints that assist sellers when creating a product listing: automatic category classification and market-based price estimation.

All endpoints are **stateless** — each request is self-contained with no memory of prior calls. Authentication is **not required** to call these endpoints.

Health Check

GET `/health`

```
{ "status": "ok" }
```

Category Suggestion

POST `/wallabot/category`

Classifies a product into one of the caller-provided categories using a LangChain LCEL chain backed by **GPT-4o-mini**.

The agent selects the most appropriate existing category. It only proposes a new category name (setting `is_new_category: true`) when none of the provided options is a good fit.

On output validation failure the agent retries up to **3 times** with progressively detailed correction feedback. On LLM provider failure it returns the closest available category with `confidence: 0.0` rather than raising a 5xx error.

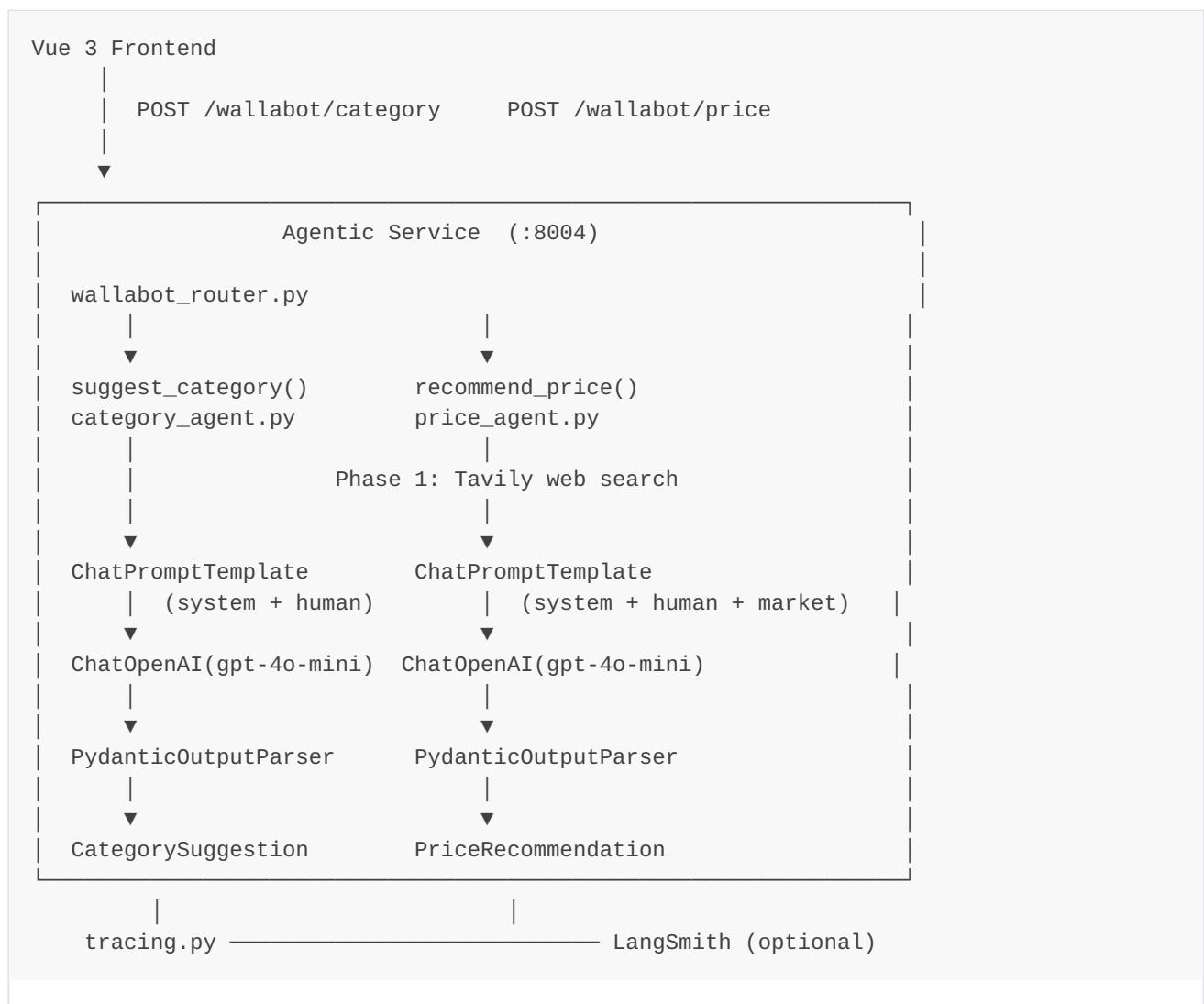
Wallabot AI Agent

The Wallabot Agentic Service (`backend/agent-service/`) is the AI subsystem of the marketplace. It is an independent FastAPI microservice that exposes two intelligent endpoints to assist sellers when creating a product listing:

- `POST /wallabot/category` — automatic product category classification
- `POST /wallabot/price` — market-based price estimation in EUR

Both are powered by [LangChain LCEL](#), OpenAI **GPT-4o-mini**, and the **Tavily** web search API. Every call is stateless — no session or memory is kept between requests.

Architecture Overview



Testing & Quality

Overview

Wallabot's test suite is organised into four distinct layers: **unit tests** (per service), **integration tests** (cross-service, live stack), **performance / load tests** (Locust), and **smoke tests** (Docker Compose health checks). Each layer has a different strategy for isolating data and a different activation mechanism.

Coverage Summary

Service	Tests	Lines covered	Coverage
auth-service	34	331 / 415	80%
inventory-service	44	267 / 301	89%
transaction-service	57	511 / 619	83%
agentic-service	75	277 / 318	87%
Total	210	1386 / 1653	84%

Coverage is measured with `pytest-cov` against each service's `app/` package. The CI pipeline enforces a minimum of 50% via `--cov-fail-under=50`.

DevOps & CI/CD

Live Production Environment

Wallabot is deployed on [Render](#) and is **publicly accessible right now**.

Service	Production URL	API docs
Frontend (app)	https://wallabot-frontend.onrender.com	—
Auth Service	https://wallabot-auth.onrender.com	https://wallabot-auth.onrender.com/docs
Inventory Service	https://wallabot-inventory.onrender.com	https://wallabot-inventory.onrender.com/docs
Transaction Service	https://wallabot-transaction.onrender.com	https://wallabot-transaction.onrender.com/docs
Agentic Service	https://wallabot-agentic.onrender.com	https://wallabot-agentic.onrender.com/docs

The interactive Swagger UI at [/docs](#) and ReDoc at [/redoc](#) are enabled on every backend service in production — useful for manual testing without running anything locally.

Note on free-tier cold starts: all services run on Render's free plan. After 15 minutes of inactivity Render spins the container down. The first request after inactivity will take 30–60 seconds while the container boots. Subsequent requests are fast.

Sphinx Documentation (GitHub Pages)

The full project documentation (this site) is automatically published to GitHub Pages on every push to [main](#):

<https://BSPQ25-26.github.io/BSPQ26-E9/sphinx/>

A downloadable PDF version of the documentation is also generated and available at:

<https://BSPQ25-26.github.io/BSPQ26-E9/sphinx/BSPQ26-E9.pdf>